

COMICCRAFT

AI Comic Story Creator

Department of computer applications

Naan mudhalvan
project
documentation

Name : K. Janani

Abstract

ComicCraft is an AI-powered comic story creation web application designed to convert a short story idea into an illustrated comic. The reference implementation uses Google Gemini for story planning, narration, dialogue and character descriptions, while AI image-generation services are used to create artwork for comic panels. The generated content is assembled into a comic layout and exported as a PDF. The application also provides a gallery, an explanatory page, and a JSON API.

Project Objective

- Create a simple platform for generating comics from a one-line story idea.
- Automate story planning, character creation, narration and dialogue.
- Generate visual artwork for each comic panel using AI image-generation services.
- Maintain visual consistency of the main character across panels through repeated character descriptions.
- Combine artwork, narration and speech bubbles into a readable comic layout.
- Allow the finished comic to be exported as a PDF.
- Provide a gallery for previously generated comics and an API endpoint for programmatic generation.

Reference Architecture

The repository is organized around a FastAPI application, HTML/Jinja templates, static assets and separate modules for Gemini interaction, image generation, layout building and PDF export. The README describes a five-step internal generation pipeline: story outline → narration/dialogue → panel artwork → layout → PDF export.

Eight-Phase Project Development Plan

Phase	Main Purpose	Key Activities	Main Deliverable
1. Brainstorming & Ideation	Define the problem and idea	Identify target users, comic-generation concept, core features and expected outcome	Project concept and initial feature list
2. Requirement Analysis	Identify what the system must do	Functional/non-functional requirements, inputs, outputs, APIs, AI services and constraints	Software Requirements Specification
3. Project Design	Design the solution	Architecture, modules, UI pages, data flow, AI pipeline and export flow	System design and interface plan
4. Project Planning	Organize implementation	Tasks, responsibilities, milestones, technologies, testing strategy and risk planning	Development plan

5. Project Development	Build the application	Implement FastAPI routes, Gemini modules, image generation, layout, PDF export, pages and configuration	Working ComicCraft application
6. Project Testing	Verify correctness and reliability	Functional, integration, UI, API, error-handling and output tests	Test report and corrected build
7. Project Documentation	Record the project	Prepare technical documentation, setup guide, architecture, user guide and limitations	Complete project documentation
8. Project Demonstration	Present the working system	Explain problem, workflow, features, live generation, output and future scope	Demo and presentation

1. Brainstorming & Ideation Phase

Purpose: The purpose of this phase is to establish the idea of an easy-to-use AI comic creator that reduces the effort required to write, illustrate and assemble a comic.

Problem Identified: Traditional comic creation can require separate skills for writing, drawing, character design, page layout and publishing. ComicCraft combines these activities into a single workflow.

Proposed Solution: The user provides a short story idea. The system uses AI to plan the story, create characters and dialogue, generate panel artwork, assemble the comic and export it.

Initial Feature Ideas:

- **One-line story input**
- **AI-generated story outline**
- **AI-generated title, characters, narration and dialogue**
- **Multiple visual styles such as anime, cartoon, watercolor and pixel art**
- **Six-panel comic generation in the reference configuration**
- **Speech bubbles and narration captions**
- **AI-generated panel artwork**
- **Character-consistency prompting**
- **PDF export with a cover page**
- **Comic gallery**
- **How-it-works page**
- **JSON API and interactive API documentation**
- **Fallback image-generation services and friendly error handling**

2. Requirement Analysis Phase

Functional Requirements:

- **The system shall accept a story idea from the user.**
- **The system shall generate a story structure and characters.**
- **The system shall generate narration and dialogue.**
- **The system shall generate artwork for comic panels.**
- **The system shall combine text and artwork into comic pages.**
- **The system shall export the completed comic as a PDF.**
- **The system shall provide a gallery of generated comics.**
- **The system shall expose a JSON comic-generation API.**
- **The system shall report setup and generation errors clearly.**

3. Project Design Phase

High-Level Architecture: User Interface → FastAPI routes → Gemini story/dialogue modules → Image generation module → Layout builder → PDF exporter → Gallery/Output.

Core Modules: The reference repository separates the application into modules such as main.py, routes.py, config.py, gemini_client.py, gemini_flash.py, gemini_pro.py, image_generator.py, layout_builder.py and exporters.py.

UI Pages: The reference README identifies pages for creating a comic, viewing the gallery, learning how the system works, API documentation and health/status checking.

Design Principle: Each major generation task is separated into a module so that the story, image, layout and export stages can be developed and tested independently.

4. Project Planning Phase

Technology Plan: Python 3.10+, FastAPI, Jinja2 templates, Google Gemini, AI image-generation services, HTML/CSS and PDF generation.

Implementation Sequence: Set up environment → configure API keys → implement story generation → implement narration/dialogue → implement image generation → implement layout → implement PDF export → connect UI → test → document.

Team Planning: The reference README lists Sai Varshini as team leader along with Sai Medha, Safiya Nawaz and Roshini.

Risk Planning: Plan for API quota limits, image-service failures, missing API keys, network problems, slow generation and inconsistent generated artwork.

5. Project Development Phase

Backend Development: Build the FastAPI application and

connect routes to the generation pipeline.

Story Generation: Use Gemini to transform the user's idea into a structured story with a title, characters and panel-level content.

Narration and Dialogue: Generate narration captions and speech-bubble dialogue for the panels.

Image Generation: Create artwork for each panel using available AI image services. The reference project uses automatic fallback between several services.

Character Consistency: Repeat the main character's descriptive information in image prompts so the character remains visually similar across panels.

Layout Generation: Combine generated panel images with narration and dialogue into a comic page layout.

PDF Export: Create a PDF containing the cover and comic pages.

Frontend: Provide the comic creation page, gallery, about/how-it-works page and export-success experience.

6. Project Testing Phase

Unit Testing: Test individual modules such as configuration, Gemini helpers, image generation, layout generation and exporters.

Integration Testing: Verify that story output correctly feeds narration generation, then image generation, layout and PDF export.

Functional Testing: Check that a user can submit a story, select a style, generate a comic, preview it and obtain the PDF.

API Testing: Test POST /generate-comic/json and confirm valid JSON responses and error handling.

UI Testing: Check navigation, form validation, gallery display, status messages and export flow.

Failure Testing: Test missing API keys, API quota

exhaustion, image-service failures, network problems and invalid inputs.

Output Testing: Check panel count, readability of captions/dialogue, character consistency, image quality and PDF opening.

7. Project Documentation Phase

Technical Documentation: Document architecture, modules, dependencies, configuration, API endpoints and generation workflow.

Installation Documentation: Explain Python installation, dependency installation, environment configuration, API-key setup and application startup.

User Documentation: Explain how to enter a story idea, choose a style, generate the comic, view the gallery and export the result.

Limitations: AI output may vary; image services can fail or reach quotas; generated characters may not be perfectly consistent; internet access is required for cloud AI services.

Security Note: API keys must remain private and should not be committed to the repository. The reference README specifically warns against uploading the .env file.

8. Project Demonstration Phase

Demo Introduction: Introduce the problem, project objective and the idea behind AI-assisted comic creation.

Live Workflow: Enter a short story idea → select a visual style → generate story and dialogue → generate panel artwork → assemble comic → preview → export PDF.

Features to Demonstrate: Story generation, character generation, panel artwork, speech bubbles, style selection, PDF export, gallery and API documentation.

Expected Result: A completed illustrated comic containing a cover, panels, narration and dialogue, available as a PDF.

Future Scope: Possible future improvements include more layout choices, richer character control, additional visual styles, editing tools, user accounts, cloud storage and improved sharing.

ComicCraft Generation Workflow

- 1. User enters a one-line story idea.
- 2. Story-planning model creates the story structure and characters.
- 3. Dialogue/narration stage creates panel-level text.
- 4. Image-generation stage creates artwork for each panel.
- 5. Layout builder combines images, captions and speech bubbles.
- 6. Exporter creates the final PDF.
- 7. The generated comic can be shown in the gallery and downloaded.

Reference Project Structure

- app/main.py – FastAPI application entry point
- app/routes.py – web pages and API endpoints
- app/config.py – settings, paths and API configuration
- app/setup_keys.py – API-key setup helper
- app/gemini_client.py – shared Gemini helper, retries and fallback
- app/gemini_flash.py – story outline stage
- app/gemini_pro.py – narration and dialogue stage
- app/image_generator.py – panel artwork generation
- app/layout_builder.py – combines text and images
- app/exporters.py – PDF export
- templates/ – Jinja2 HTML templates

- `static/css/style.css` – styling
- `static/fonts/` – PDF fonts
- `static/panels/` – generated panel images at runtime
- `static/exports/` – generated PDF files at runtime
- `requirements.txt` – Python dependencies
- `.env.example` – API-key configuration template

Expected Inputs and Outputs

Category	Input	Output
Story	One-line story idea	Story outline, title and characters
Style	Selected visual style	Style-specific panel prompts/artwork
Characters	AI-generated character descriptions	Repeated descriptions for visual consistency
Dialogue	Story context	Narration captions and speech bubbles
Artwork	Panel prompts	Panel images
Layout	Images + text	Comic pages
Export	Completed comic	PDF comic
API	JSON request	JSON comic-generation response

Conclusion

ComicCraft demonstrates an end-to-end AI-assisted content-generation workflow in which natural-language input is transformed into a structured and illustrated comic. For an academic software project, the eight phases above can be used as the project-development chapter structure: ideation defines the concept, requirements define what must be built, design defines how it will work, planning organizes implementation, development produces the system, testing verifies it, documentation records it, and demonstration communicates the final result.

Reference

Primary reference: safiyanawaz07/comiccraft – ComicCraft: AI Comic Story Creator (GitHub repository). The project details in this document are based on the repository README and visible project structure.